UANL
UNIVERSIDAD AUTÓNOMA DE NUEVO LEÓN

FIME
FACULTAD DE INGENIERÍA MECÁNICA Y ELÉCTRICA

UNIVERSIDAD AUTÓNOMA DE NUEVO LEÓN

FACULTAD DE INGENIERÍA MECÁNICA Y ELÉCTRICA

Diccionario de datos

Ascenso a Root

PIA

Sistemas Operativos

Equipo 1

Grupo:003 | Hora: Martes V4-V6 | Aula: 1303

Profesora: Dra. Marín Martínez Norma Edith.

Integrantes del equipo

Matrícula	Nombre	Programa	Foto	Porcentaje de participación
2144053	Lozano Garza Camila del Carmen	IAS		100%
2160859	Vera Moreno Oscar Elias	ITS		100%
2226699	Sánchez Santiago Osmar Abdi	ITS		100%
2178081	Cisneros Moreno Valeria Nohemí	ITS		100%
2144813	Torres Guerrero David Alonso	ITS		100%
2065934	Gonzalez Belloc Yael Ricardo	ITS		100%
2161066	Ramirez Fabian Maria de los Angeles	IAS		100%
2169669	Sánchez Juárez Emiliano	ITS		100%

Matrícula	Nombre	Programa	Foto	Porcentaje de participación
2097304	Jaime Sebastian Esparza Leyva	ITS		100%
2158392	Martinez Tamez Julio César	IAS		100%
2135792	Muñoz Hernández Salvador	IAS		100%
1320176	Maycott López Gilberto	IAS		100%

1. Alcance y panorama general

Ascenso a Root no usa base de datos, servidor ni conexión a Internet. Es una app Android híbrida: Kotlin abre un WebView que muestra una interfaz HTML/CSS/JavaScript local. Por eso los datos se distribuyen en cinco almacenes, descritos en este diccionario:

Almacén	Ubicación	¿Persiste al cerrar la app?	Contenido
Progreso y preferencias	localStorage del WebView, clave ascenso-a-root-v1	Sí	Misiones completadas, sonido, avisos
Estado de sesión	Objeto state en memoria (app.js)	No	Pantalla actual, paso, líneas de la terminal
Catálogos constantes	Constantes en app.js: missions, teamMembers, activities	Sí (van dentro del código)	Contenido de misiones, integrantes, AFs
Datos de la capa Android	Kotlin: Intent extras, caché de páginas, configuración Gradle y Manifest	Parcial	Datos del visor de PDF y configuración de la app
Recursos empaquetados	Carpeta assets/	Sí	Imágenes, PDF, HTML, CSS, JS

2. Almacenamiento persistente

Esta sección describe los únicos datos que la app conserva después de cerrarse. Se guardan en el almacenamiento local del WebView, en el propio dispositivo, sin enviarse a ningún servidor.

2.1 Entidad Progreso (localStorage)

Progreso es el registro que guarda qué misiones terminó el usuario y sus dos preferencias. Se almacena como un solo objeto JSON bajo la clave ascenso-a-root-v1. Se lee con loadData() al abrir la app y se escribe con saveData() al completar una misión, al cambiar un interruptor o al restablecer el progreso.

Campo	Tipo	Valores permitidos	Valor por defecto	Descripción
completed	Arreglo de enteros	1 a 5, sin repetidos, ordenado de menor a mayor	[]	Identificadores (id) de las misiones terminadas. Al leer se descartan los valores que no sean enteros entre 1 y 5.
sound	Booleano	true / false	true	Activa o silencia el tono breve. Solo vale false si así se guardó.
notices	Booleano	true / false	true	Activa o quita la frase «Puedes tocar Ver pista» en mensajes de error genéricos.

Ejemplo de contenido guardado después de terminar las misiones 1 y 2: {"completed":[1,2],"sound":true,"notices":true}

Reglas importantes:

- Si el JSON guardado está dañado, loadData() captura el error y regresa los valores por defecto.
- Restablecer progreso vacía únicamente completed; sound y notices se conservan.
- Una misión repetida no se vuelve a agregar a completed, por eso el XP no aumenta al repetir.

2.2 Datos derivados (se calculan, no se guardan)

Los datos derivados son valores que la app calcula cada vez que dibuja una pantalla a partir de completed. No se almacenan, por lo que siempre son consistentes con el progreso guardado.

Dato	Fórmula en el código	Rango	Dónde se muestra
Porcentaje de avance	round(completed.length / 5 * 100)	0, 20, 40, 60, 80, 100	Dashboard (círculo) y encabezado del Mapa
Misiones completadas	completed.length	0 a 5	Dashboard (tarjeta «Misiones», formato n/5)
Puntos XP	completed.length * 100	0 a 500	Dashboard (tarjeta «Puntos»)
Misiones desbloqueadas	completed.length + 1	1 a 6	Mapa: una misión con id mayor queda bloqueada
Siguiente misión	Primera misión cuyo id no está en completed; si todas están completas, la misión 5	Índice 0 a 4	Dashboard (Continuar ruta) y botón del Mapa
Ruta completa	completed.length === 5	Booleano	Cambia textos de Dashboard y Mapa

3. Estado de sesión (memoria)

El estado de sesión es el objeto state de app.js. Guarda lo que el usuario está haciendo en este momento (pantalla abierta, paso de la misión, texto de la terminal) y se pierde al cerrar la app.

Campo	Tipo	Valores	Valor inicial	Descripción
screen	Cadena	cover, home, dashboard, map, mission, settings, team, afs, credits	cover	Pantalla visible.
mission	Entero	0 a 4	0	Índice de la misión abierta dentro de missions.

Campo	Tipo	Valores	Valor inicial	Descripción
step	Entero	0 hasta steps.length	0	Paso actual de la misión. Cuando es igual a steps.length, la misión está terminada.
lines	Arreglo de Línea	Ver tabla siguiente	[]	Registro mostrado en la terminal simulada.
hint	Booleano	true / false	false	Indica si se muestra la pista del paso actual.
confirmReset	Booleano	true / false	false	Muestra el cuadro «¿Restablecer progreso?».
acceptedCommands	Arreglo de cadenas	Comandos ya aceptados	[]	Sirve para avisar si el usuario repite un comando que ya completó un paso.
copyName	Cadena o nulo	Nombre de archivo válido	null	Nombre de la copia creada con cp en la misión 4; lo usan la instrucción y la pista del paso 2.
routeJustCompleted	Booleano	true / false	No existe hasta abrir una misión	Se activa al terminar la quinta misión; muestra el botón «Ver conclusión» y abre los créditos a los 2.5 segundos.

La entidad Línea de terminal representa cada renglón del registro de la terminal simulada y es el tipo de cada elemento de state.lines:

Campo	Tipo	Valores	Descripción
kind	Cadena	command, output, error	Define el estilo: comando escrito (con prefijo $), respuesta simulada o mensaje de corrección.
text	Cadena	Texto libre	Contenido de la línea. Se escapa con safe() antes de mostrarse.

Al abrir una misión, openMission() reinicia step, lines, hint, acceptedCommands, copyName y routeJustCompleted. Por eso salir de una misión incompleta y volver a entrar la reinicia desde el primer paso.

4. Catálogos constantes

Los catálogos son datos fijos escritos dentro de app.js. No cambian durante el uso de la app; definen el contenido de las misiones, la lista de integrantes y las actividades fundamentales.

4.1 Misión (missions, 5 registros)

Una misión es cada nivel de la ruta de aprendizaje. Esta tabla describe los campos que tiene cada uno de los cinco registros.

Campo	Tipo	Descripción
id	Entero (1 a 5)	Identificador. Es el valor que se guarda en completed.
title	Cadena	Nombre de la misión.
commands	Cadena	Comandos que se practican; se muestra en Mapa y Dashboard.
icon	Cadena (símbolo)	Símbolo del módulo en el Mapa cuando está disponible. Se reemplaza por un acierto paloma si está completa y por un candado si está bloqueada.
objective	Cadena	Tarea concreta que se pide.
lesson	Cadena	Explicación breve del comando.
steps	Arreglo de Paso	Pasos de la misión (1 o 2).
ending	Cadena	Mensaje educativo del panel de misión completada.

4.2 Paso (steps, elemento de una misión)

Un paso es cada instrucción individual que el usuario debe resolver dentro de una misión. Cada paso corresponde a un comando que se escribe en la terminal.

Campo	Tipo	Descripción
command	Cadena	Forma canónica del comando esperado.
instruction	Cadena	Texto «Paso n de m» que indica qué hacer.
hint	Cadena	Texto que aparece al tocar «Ver pista».
output	Cadena	Respuesta simulada que escribe la terminal al aceptar el comando.

4.3 Contenido de las misiones

Esta tabla resume los valores reales de los cinco registros de missions: qué comandos se practican y cuántos pasos tiene cada nivel.

id	Título	Comandos	Icono	Pasos	Objetivo
1	Bienvenida a la terminal	pwd	›	1	Descubre en qué carpeta estás.

id	Título	Comandos	Icono	Pasos	Objetivo
2	Navegar directorios	ls · cd	⌖	2	Consulta el contenido de tu carpeta y entra en Documentos.
3	Crear archivos y carpetas	mkdir · touch	+	2	Primero crea la carpeta proyectos; después, crea el archivo notas.txt.
4	Copiar y mover elementos	cp · mv	↗	2	Haz una copia de notas.txt y mueve esa copia a proyectos.
5	Administrador supremo	sudo	✦	1	Actualiza la lista de paquetes con permisos de administrador.

4.4 Comandos aceptados por paso

Esta tabla define qué texto acepta la terminal simulada en cada paso y qué respuesta devuelve. Antes de validar, el texto escrito se recorta (trim) y los espacios repetidos se reducen a uno. La comparación distingue mayúsculas de minúsculas y una entrada vacía se ignora.

Misión / paso	Comando canónico	Variantes que también acepta	Salida simulada
1 / 1	pwd	Ninguna	/home/explorador
2 / 1	ls	Ninguna	Documentos Descargas Imágenes
2 / 2	cd Documentos	cd ./Documentos, cd ~/Documentos, cd /home/explorador/Documentos	Ahora estás en /home/explorador/Documentos
3 / 1	mkdir proyectos	mkdir ./proyectos, mkdir proyectos/	Carpeta proyectos creada.
3 / 2	touch notas.txt	touch ./notas.txt	Archivo notas.txt creado.
4 / 1	cp notas.txt copia.txt	Cualquier nombre de copia que cumpla, distinto de notas.txt y que no apunte a proyectos; el origen puede escribirse ./notas.txt	Copia creada: <nombre>
4 / 2	mv <copia> proyectos	Destino proyectos/, ./proyectos o proyectos/<archivo sencillo>; el origen debe ser la copia, nunca notas.txt	<nombre> se movió a <destino>.
5 / 1	sudo apt update	sudo apt-get update	Lista de paquetes actualizada (simulación).

Cuando el comando no coincide, la app responde con un mensaje de corrección (línea de tipo error) y el paso no avanza. Hay mensajes específicos para: falta de espacio después de cd, mkdir notas.txt, mkdir touch ..., touch antes de crear la carpeta, mv antes de cp, destino proyecto sin la «s», comando ya aceptado antes y origen incorrecto.

4.5 Integrante (teamMembers, 10 registros)

Cada integrante del equipo es un registro de teamMembers. La primera tabla describe sus campos y la segunda lista los registros que contiene la app.

Campo	Tipo	Descripción
id	Cadena	Matrícula.
name	Cadena	Nombre completo, apellidos primero.
program	Cadena	Programa educativo: ITS o IAS.
photo	Cadena	Ruta relativa de la foto en assets/equipo/.
leader	Booleano (opcional)	true solo en el líder del equipo; agrega la etiqueta «Líder».

Matrícula	Nombre	Programa	Foto	Líder
2178081	Cisneros Moreno Valeria Nohemí	ITS	equipo/04.png	No
2097304	Esparza Leyva Jaime Sebastian	ITS	equipo/08.png	No
2144053	Lozano Garza Camila del Carmen	IAS	equipo/01.png	No
2158392	Martinez Tamez Julio César	IAS	equipo/09.png	No
1320176	Maycott López Gilberto	IAS	equipo/10.png	No
2161066	Ramirez Fabian Maria de los Angeles	IAS	equipo/06.png	No
2169669	Sánchez Juárez Emiliano	ITS	equipo/07.png	No
2226699	Sánchez Santiago Osmar Abdi	ITS	equipo/03.png	No
2144813	Torres Guerrero David Alonso	ITS	equipo/05.png	Sí
2160859	Vera Moreno Oscar Elias	ITS	equipo/02.png	No

La lista está ordenada alfabéticamente por apellido y se usa en dos pantallas: Equipo (con foto) y Créditos (solo nombres).

4.6 Actividad fundamental (activities, 4 registros)

Una actividad fundamental (AF) es cada trabajo del semestre cuyo PDF se puede abrir desde la app. La primera tabla describe los campos y la segunda los cuatro registros.

Campo	Tipo	Descripción
id	Cadena	Clave de la AF (AF1 a AF4).
title	Cadena	Tema de la actividad.
date	Cadena	Fecha de entrega, en texto.
pages	Entero	Número de páginas declarado.

Campo	Tipo	Descripción
file	Cadena	Ruta del PDF dentro de assets/.

id	Título	Fecha de entrega	Páginas	Archivo
AF1	Arquitectura, desempeño y tipo de un sistema operativo	11 de agosto de 2026	37	afs/AF1.pdf
AF2	La concurrencia en los sistemas operativos	18 de agosto de 2026	29	afs/AF2.pdf
AF3	Almacenaje, memoria y archivos	15 de septiembre de 2026	23	afs/AF3.pdf
AF4	Redes y seguridad en sistemas distribuidos	13 de octubre de 2026	42	afs/AF4.pdf

Se verificó que el número de páginas declarado coincide con el de cada PDF empaquetado.

4.7 Pantallas y acciones de la interfaz

La interfaz se organiza en nueve pantallas, identificadas por el valor de state.screen. La tabla indica qué muestra cada una y qué botón de la barra inferior queda resaltado.

Valor de screen	Pantalla	Barra inferior activa
cover	Portada de la entrega	Sin barra
home	Inicio (astronauta y botón «Comenzar expedición»)	Sin barra
dashboard	Dashboard: introducción PIA, progreso, XP, continuar ruta	Dashboard
afs	Lista de actividades fundamentales	AFs
map	Mapa de misiones	Mapa
mission	Misión con terminal simulada	Mapa
team	Integrantes	Equipo
settings	Ajustes	Ajustes
credits	Conclusión y créditos	Ajustes

Las acciones (data-action) son los eventos que disparan los botones de la interfaz. Esta tabla muestra qué datos modifica cada una:

Acción	Efecto sobre los datos
cover-continue	screen = home
start	screen = dashboard

Acción	Efecto sobre los datos
nav	Cambia screen y desactiva routeJustCompleted
open-mission	Reinicia el estado de misión y abre la misión indicada, solo si no está bloqueada
hint	hint = true
toggle-sound / toggle-notices	Invierte la preferencia y la guarda en localStorage
confirm-reset / cancel-reset	Muestra u oculta el cuadro de confirmación
reset	completed = [] y guarda; conserva sound y notices
open-credits	screen = credits

5. Datos de la capa Android (Kotlin)

La parte nativa de la app está escrita en Kotlin y tiene dos actividades: MainActivity, que aloja el WebView, y PdfViewerActivity, que muestra los PDF de las AFs. Esta sección describe los datos que manejan.

5.1 Datos que viajan entre actividades (Intent)

Un Intent extra es un dato que una actividad Android entrega a otra al abrirla. Cuando el usuario toca una AF, MainActivity intercepta el enlace al PDF y abre PdfViewerActivity con estos dos datos:

Clave	Constante	Tipo	Ejemplo	Descripción
asset	EXTRA_ASSET	Cadena (obligatoria)	afs/AF4.pdf	Ruta del PDF relativa a assets/. Si falta, se muestra un aviso y la actividad se cierra.
titulo	EXTRA_TITLE	Cadena	AF4 · Redes y seguridad en sistemas distribuidos	Título que se muestra en el visor; si no llega, usa «Documento».

El enlace tiene la forma afs/AF4.pdf?titulo=<texto codificado>. Solo se intercepta si el host es appassets.androidplatform.net, la ruta empieza con /assets/ y termina en .pdf.

5.2 Datos internos del visor de PDF

El visor de PDF usa objetos de Android para abrir y dibujar las páginas sin necesidad de Internet ni de otra app. Estos son los datos que mantiene mientras está abierto:

Elemento	Tipo	Descripción
renderer	PdfRenderer	Dibuja las páginas del PDF abierto.

Elemento	Tipo	Descripción
descriptor	ParcelFileDescriptor	Archivo abierto en modo solo lectura. Se cierra en onDestroy.
pageCache	LruCache<Int, Bitmap>	Caché de páginas ya dibujadas. Su tamaño es un sexto de la memoria máxima disponible.
Archivo temporal	Archivo en cacheDir/pdf/	Copia del PDF de assets/, porque PdfRenderer necesita un archivo.
total	Entero	Total de páginas; se muestra como «Página n de total».

5.3 Configuración del WebView

El WebView es el componente que muestra la interfaz web dentro de la app. Su configuración define qué puede hacer y qué se le prohíbe:

Parámetro	Valor	Efecto
URL inicial	https://appassets.androidplatform.net/assets/index.html	Los archivos de assets/ se sirven con WebViewAssetLoader, sin Internet.
javaScriptEnabled	true	Necesario para ejecutar app.js.
domStorageEnabled	true	Permite usar localStorage.
allowFileAccess	false	Bloquea el acceso directo a archivos.
allowContentAccess	false	Bloquea el acceso a proveedores de contenido.
Botón Atrás	Llama a window.goBackFromAndroid()	Cierra el diálogo, o regresa de misión a mapa, de créditos a ajustes, o de otra pantalla a inicio; en portada e inicio no hace nada.

5.4 Configuración de la aplicación

Estos parámetros identifican y configuran la app como paquete Android (compilación, versión, tema y permisos):

Parámetro	Valor	Archivo
applicationId / namespace	com.equipo.ascensoaroot	app/build.gradle.kts

Parámetro	Valor	Archivo
compileSdk / targetSdk	35	app/build.gradle.kts
minSdk	26 (Android 8.0)	app/build.gradle.kts
versionCode / versionName	1 / 1.0	app/build.gradle.kts
Java / Kotlin jvmTarget	17	app/build.gradle.kts
Dependencias	appcompat 1.7.0, core-ktx 1.15.0, activity-ktx 1.9.3, webkit 1.12.1	app/build.gradle.kts
Nombre visible (app_name)	Ascenso a Root	res/values/strings.xml
Tema	Theme.UbuntuMision, barras de estado y navegación #171419	res/values/styles.xml
Orientación	Solo vertical (portrait) en ambas actividades	AndroidManifest.xml
Teclado	adjustResize en MainActivity	AndroidManifest.xml
Permisos	Ninguno declarado	AndroidManifest.xml
allowBackup	true	AndroidManifest.xml

6. Recursos empaquetados (assets/)

Los recursos empaquetados son los archivos que se instalan dentro de la app (código de la interfaz, imágenes y PDF). Permiten que funcione sin conexión.

Recurso	Tipo	Uso
index.html	HTML	Punto de entrada: contenedor #app y carga de styles.css y app.js.
styles.css	CSS	Colores, tipografía, tarjetas, mapa, terminal y barra inferior.
app.js	JavaScript	Pantallas, validación de comandos, progreso y ajustes.
astronauta.png	Imagen	Ilustración de la pantalla Inicio.
app_logo.png	Imagen	Logotipo de la pantalla Ajustes.
logo-uanl.png, logo-fime.png	Imágenes	Logos de la portada.
equipo/01.png a 10.png	Imágenes	Fotos de los integrantes (campo photo).
afs/AF1.pdf a AF4.pdf	PDF	Actividades fundamentales que abre el visor (campo file).
icons/home.png, docs.png, map.png, users.png, settings.png	Imágenes	Iconos de la barra inferior (clases ic-*). También existe una copia de cada uno en la raíz de assets/.

Recurso	Tipo	Uso
logo-app-astronauta.png	Imagen	Presente en el proyecto; app.js no la referencia.
res/drawable-nodpi/app_logo.png	Imagen	Ícono de la app en el teléfono (fuera de assets/).

7. Validación y protección de los datos

Esta tabla reúne los controles que usa la app para evitar datos inválidos y proteger la información del usuario.

Control	Dónde	Descripción
Escape de HTML	safe() en app.js	Convierte & < > " ' en entidades antes de insertar texto de usuario o de catálogo con innerHTML.
Normalización de comandos	runCommand()	trim, espacios repetidos reducidos a uno, sensible a mayúsculas.
Nombre de la copia	checkCommand()	Expresión regular que solo permite letras, números, punto, guion y guion bajo, empezando con letra o número.
Progreso guardado	loadData()	Filtra valores fuera de 1 a 5 y recupera valores por defecto si el JSON está dañado.
Bloqueo de misiones	openMission()	Rechaza índices inválidos y misiones cuyo id supere completed.length + 1.
Confirmación destructiva	confirmReset	El progreso solo se borra después de confirmar en el cuadro.
Aislamiento de la terminal	Diseño de la app	Los comandos se comparan con texto; nunca se ejecutan en Android ni en Ubuntu.
Archivos locales	WebView	allowFileAccess y allowContentAccess desactivados.

Privacidad: la app no pide datos al usuario ni declara permisos. Lo único que se guarda del usuario es el progreso (misiones completadas) y dos preferencias, todo en el dispositivo. Los nombres y matrículas del equipo son datos fijos del código, no se capturan. Como allowBackup está en true, el respaldo automático de Android podría incluir ese almacenamiento local.

8. Relaciones entre los datos

Aunque la app no tiene base de datos relacional, sus datos se relacionan entre sí por identificadores, índices y rutas de archivo. La tabla indica cómo se conecta cada dato con otro.

Origen	Destino	Tipo de relación
Progreso.completed[n]	missions[].id	Cada elemento hace referencia a una misión existente (1 a 5).
state.mission	missions[índice]	Índice de la misión abierta.
state.step	missions[].steps[índice]	Paso actual de esa misión.
state.copyName	Paso 2 de la misión 4	Se genera en el paso 1 y personaliza la instrucción y la pista del paso 2.
teamMembers[].photo	assets/equipo/*.png	Ruta de la foto.
activities[].file	assets/afs/*.pdf	Ruta del PDF.
activities[].id + title	Intent extra titulo	Se concatenan con « · » y viajan en el enlace al visor.
Intent extra asset	assets/afs/*.pdf	El visor copia ese archivo a la caché y lo abre con PdfRenderer.